UTN FRLP

Desarrollo de Software 2026

Docker + Previa Backend

Clase #13 — 6 de julio de 2026

Matías Corti - Martina García Améndola



¡Cerraron el front!

Entregaron una aplicación React real, no un ejercicio de juguete:

✅ Componentes propios, tipados con TypeScript

✅ Navegación entre páginas (routing)

✅ Formularios con validación (Zod)

✅ Consumo de datos (fetch + useFetch)

✅ Trabajo en equipo, con Git y PRs

Hace 3 meses muchos no habían escrito una línea de React. No es poco!



Temas de hoy

- 🐳 Docker: teoría + demo en vivo
- 🧩 Del mock al modelo: cómo pensamos el backend
- 🛠️ Taller de planning por grupo: modelo de datos + endpoints
- 🎯 Se abre el Checkpoint 2 (individual) — tu Librería React · cierra en C15 (3/8)
- 🏖️ Receso invernal: 20/7 al 1/8 · C14 (13/7) es la última clase antes



Objetivos: Docker

Hoy con Docker vamos a:

- Entender el problema de los entornos ("funciona en mi máquina")
- Diferenciar imagen vs contenedor
- Conocer los comandos esenciales para usar contenedores
- Leer un Dockerfile (la receta de una imagen)
- Usar Docker Compose para levantar varios servicios juntos

El problema de los entornos





El problema de los entornos

¿Por qué? Cada máquina es un mundo distinto:

"En mi máquina anda" 🤷

→ en la del compañero, no.

Máquina Dev 1	Máquina Dev 2	Servidor Producción
Node 18.17.0	Node 22.14.1	Node 20.11.3
PostgreSQL 14	PostgreSQL 13	PostgreSQL 15
Windows 11	macOS Sonoma	Ubuntu 22.04
npm 9.6.7	npm 8.3.1	npm 10.2.4
Python 3.9	No tiene Python	Python 3.11
Variables locales	Variables diferentes	¿Qué variables?

¿Cómo garantizamos que funcione igual en todos lados?



Así como los contenedores revolucionaron el transporte marítimo estandarizando el empaque, Docker estandariza el software



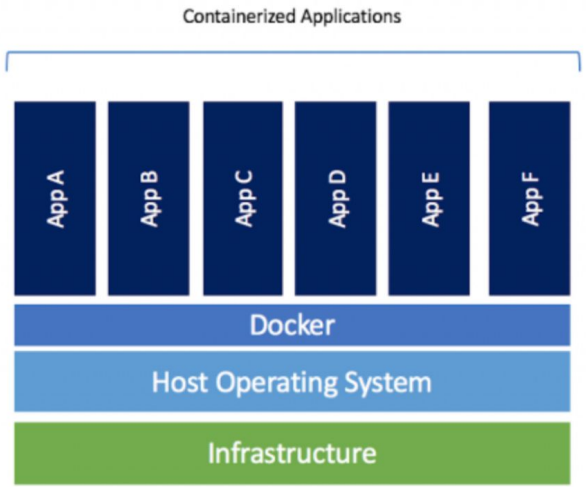
¿Qué es Docker?

Es una plataforma de contenerización: **permite empaquetar una aplicación junto con todas sus dependencias** (librerías, configuraciones, binarios, variables de entorno, etc) dentro de una imagen **inmutable.**

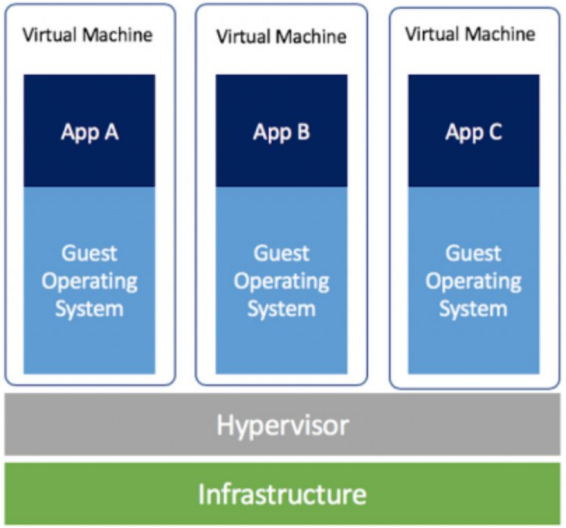
¿Por qué?
A partir de esa imagen se crean contenedores: instancias aisladas y reproducibles que se ejecutan de la misma manera en cualquier máquina que tenga Docker.

✅ **Clave:** el entorno de ejecución es idéntico en desarrollo, testing y producción.

CONTENEDORES vs MÁQUINAS VIRTUALES



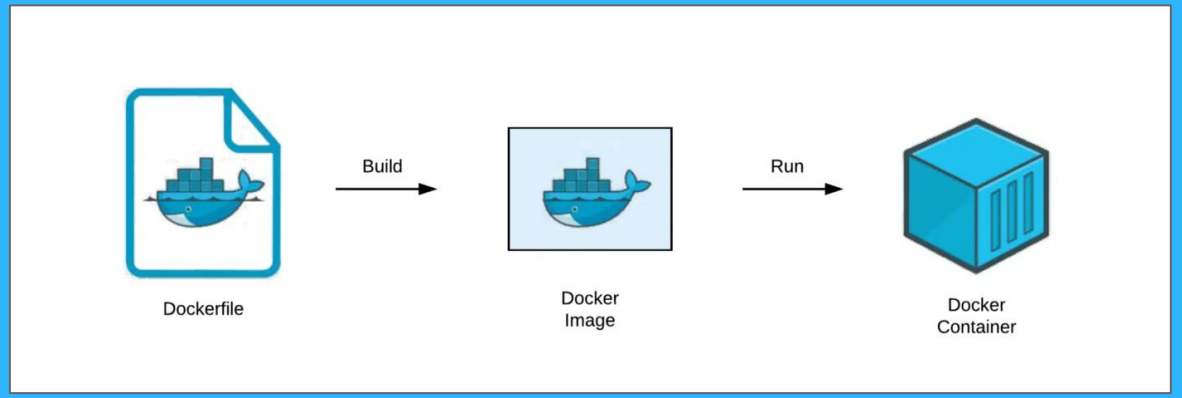
- Virtualiza el OS
- Comparte el kernel del OS
- Liviano (MBs)
- Segundos para iniciar
- Eficiente en recursos



- Virtualiza el hardware
- Sistema Operativo Completo
- Pesada (GBs)
- Minutos para iniciar
- Mayor consumo de recursos



CONCEPTOS CLAVE



Dockerfile	Archivo de texto que define **pasos de construcción** de una imagen (sistema base, dependencias, configuración y comando de inicio).
Imagen *Image*	Resultado **inmutable** de la construcción. Contiene el sistema de archivos y las dependencias exactas. Se versiona y se puede distribuir.
Contenedor *Container*	**Instancia** en ejecución de una imagen. Proceso aislado que usa el kernel del host pero su propio sistema de archivos.



DOCKERFILE

LA RECETA

📑 **Idea de Capas**
Cada instrucción es una capa cacheada, se regenera sólo si algo cambia, no ejecuta todo cada vez

Dockerfile

```
# Imagen base: punto de partida mínimo con Node.js
FROM node:22-alpine

# Directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiar sólo archivos de dependencias
COPY package*.json ./

# Instalar dependencias en la imagen
RUN npm install

# Copiar el resto del código fuente
COPY . .

# Exponer el puerto que la app escucha
EXPOSE 3000

# Comando por defecto al iniciar el contenedor
CMD ["npm", "run", "dev"]
```

Indica el sistema de archivos inicial. `alpine` es una distribución Linux mínima, ideal para imágenes livianas y seguras.

Crea y establece `/app` como carpeta raíz de las operaciones siguientes. Evita rutas absolutas en los comandos.

Copiamos solo los archivos de dependencias para que la capa de instalación quede cacheada; si solo cambia el código, Docker no reinstala todo.

Ejecuta el comando dentro del contenedor. En este caso ejecuta el install para incorporar `node_modules`.

Ahora se copian los archivos de la aplicación. Esta capa se vuelve a crear solo si cambian los archivos copiados.

Documenta el puerto en el que la aplicación escuchará. Es informativo; el mapeo real se cuando se inicie el contenedor.

Indica el proceso que mantiene vivo el contenedor. En este caso ejecuta un comando definido en `package.json`



Docker Image



Docker Container

Imagen vs Contenedor

Analogía con Paradigma Orientado a Objetos:

- **Imagen** ≈ **clase**
- **Contenedor** ≈ **instancia** de esa clase

	Imagen	**Contenedor**
Qué es	Molde inmutable: app + deps + config	Instancia en ejecución de una imagen
Estado	No cambia; se versiona y se comparte	Vive, corre, se puede frenar y borrar
Cuántos	Una	Muchos, desde la misma imagen



Imágenes oficiales

"No reinventes la rueda"

Docker Hub (hub.docker.com) es el repositorio público de imágenes oficiales.

Bases de datos, lenguajes y herramientas comunes ya tienen imágenes mantenidas por sus desarrolladores o por la comunidad oficial.

Servicio	Imagen oficial	Comando de ejemplo
PostgreSQL	`postgres:16-alpine`	`docker run -d -p 5432:5432 -e POSTGRES_PASSWORD=pass postgres:16-alpine`
MySQL	`mysql:8.0`	`docker run -d -p 3306:3306 -e MYSQL_ROOT_PASSWORD=pass mysql:8.0`
Redis	`redis:7-alpine`	`docker run -d -p 6379:6379 redis:7-alpine`
Nginx	`nginx:stable-alpine`	`docker run -d -p 80:80 nginx:stable-alpine`



COMANDOS ESCENCIALES

Comando	¿Qué hace?	Ejemplo real
`docker build -t mi-app .`	Crea una imagen	Construye tu app
`docker run -p 3000:3000 mi-app`	Ejecuta un contenedor	Levanta tu app
`docker ps`	Lista contenedores activos	Ver qué está corriendo
`docker stop <id>`	Detiene un contenedor	Parar tu app
`docker images`	Lista imágenes	Ver qué tenés construido
`docker logs <id> -f`	Ver logs del contenedor	Debuggear problemas

En la práctica casi siempre vamos a usar la versión Compose de estos comandos (próximas slides).

Docker Compose

El Orquestador

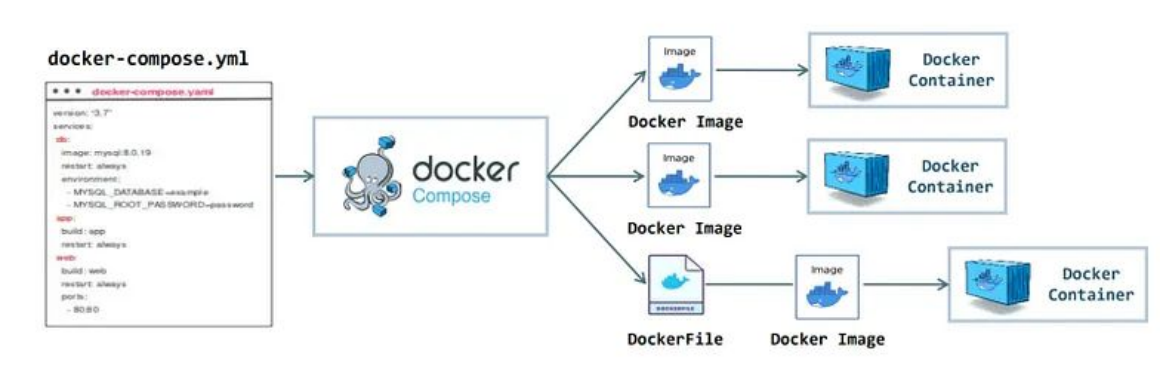
Problema

¿Si necesito Frontend + Backend + Database?

Solución

Docker Compose orquesta múltiples contenedores

- 🎻 Frontend (React)
- 🎺 Backend (Express.js)
- 🥁 Database (PostgreSQL)





Docker Compose

El Orquestador

Orquestador: levanta varios contenedores juntos con un solo archivo.

¿Por qué?
Nuestra app va a ser backend (api) + base de datos (db).
Con Compose, un comando levanta todo.
📁 docker-compose.yml archivo en la raíz del repositorio

```yaml
services:
  api:                  # nuestro backend (hello-world por ahora)
    build: ./backend
    ports: ["3000:3000"]
    env_file: [backend/.env]
    depends_on: [db]
  db:                   # PostgreSQL
    image: postgres:16-alpine
    ports: ["5432:5432"]
    env_file: [backend/.env]
    volumes:
      - pgdata:/var/lib/postgresql/data   # persistencia de datos
volumes:
  pgdata:
```



Docker Compose Comandos

Los que vas a usar todos los días:

✍️ Es docker compose (con espacio, v2). El viejo docker-compose (con guion) está discontinuado.

```bash
# CONSTRUIR: Crear las imágenes
docker compose build
# O para rebuild forzado:
docker compose build --no-cache

# LEVANTAR: Iniciar todos los servicios
docker compose up
# O en background (detached):
docker compose up -d

# VER LOGS: Para debugging
docker compose logs -f api
# O todos los servicios:
docker compose logs -f

# EJECUTAR COMANDOS: Terminal dentro de un contenedor
docker compose exec api sh

# DETENER: Parar todo
docker compose down
# O con limpieza total (¡borra la DB!):
docker compose down -v

# REINICIAR: Un servicio específico
docker compose restart api
```



Docker

Instalación



ESTRUCTURA

```
mi-proyecto/
├── 📁 frontend/
│   ├── 🐳 Dockerfile
│   ├── 📦 package.json
│   ├── .dockerignore
│   ├── 🔒 .env.example
│   ├── 🔐 .env              # no va a Git!
│   └── 📁 src/
├── 📁 backend/
│   ├── 🐳 Dockerfile
│   ├── 📦 package.json
│   ├── .dockerignore
│   ├── 🔒 .env.example
│   ├── 🔐 .env              # no va a Git!
│   ├── 📁 prisma/
│   └── 📁 src/
├── 🎼 docker-compose.yml
├── 📝 README.md
└── .gitignore
```



Docker

Instalación

Instalar **Docker Desktop** siguiendo las instrucciones

📄 frontend/.dockerignore y backend/.dockerignore

```bash
node_modules
npm-debug.log
.env
.git
.gitignore
README.md
.vscode
.idea
dist
build
.DS_Store
```



Docker

Instalación

¿Qué es? Configuración que vive fuera del código (puertos, credenciales, URLs).

- 🔒 **.env** → valores reales. **NO** va a Git (está en .gitignore)
- 📋 **.env.example** → la plantilla con las claves (sin secretos). **SÍ** va a Git, se usa como plantilla

📄 backend/.env y backend/.env.example

```bash
NODE_ENV=development
PORT=3000
FRONTEND_URL=http://localhost:5173

POSTGRES_USER=postgres
POSTGRES_PASSWORD=tu_password_segura
POSTGRES_DB=libreria_db
POSTGRES_PORT=5432
DATABASE_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:${POSTGRES_PORT}/${POSTGRES_DB}
```

📄 frontend/.env y frontend/.env.example

```bash
NODE_ENV=development
# Otras Variables
```



Docker

Instalación

📁 backend/Dockerfile

dockerfile

```
# Backend Dockerfile
FROM node:22-alpine

# Directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiar solo archivos de dependencias
COPY package*.json ./

# Instalar dependencias en la imagen
RUN npm install

# Copiar el resto del código
COPY . .

# Exponer puerto del servidor
EXPOSE 3000

# Comando por defecto al iniciar el contenedor
CMD ["npm", "run", "dev"]
```



Docker

Instalación

📁 docker-compose.yml

```yaml
services:
  api:
    build: ./backend
    ports:
      - "3000:3000"
    env_file:
      - backend/.env
    depends_on:
      - db

  db:
    image: postgres:16-alpine
    ports:
      - "5432:5432"
    env_file:
      - backend/.env
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

🔁

Mini-cierre: Docker

Lo que nos llevamos:

Docker no cambia cómo programás.

Cambia **dónde** corre tu programa: igual para todos.

Concepto	**En una línea**
Imagen	**Molde** inmutable (app + deps)
Contenedor	**Instancia** en ejecución de una imagen
Dockerfile	La **receta** de la imagen
Compose	Levanta **varios contenedores** juntos
Variables de Entorno	**.env** Config afuera del código (no va a Git) **.env.example** Template de la Config (va a Git)



Demostración en Vivo



Demo: branch C13

Vamos a levantar el ejemplo. Qué está en el branch C13

Pasos

```
git clone https://github.com/martinagarcia05/DS26-Libreria.git
cd DS26-Libreria
git checkout C13

# 1. preparar variables (modificar .env si hace falta)
cp backend/.env.example backend/.env
# 2. levantar api + db
docker compose up
# 3. abrir el navegador en http://localhost:3000
# 4. ver los dos contenedores
docker compose ps
# 5. ver los logs del backend
docker compose logs -f api
# 5. consola en el contenedor
docker compose exec api sh
# 6. bajar todo
docker compose down

# La magia del volumen (persistencia):
#   down → up       los datos de la DB siguen ahí
#   down -v         borra el volumen → la DB arranca vacía
```

Ejemplo Completo: DS26-Libreria (C13)



Previa del Backend



Del mock al modelo

Con los **mocks**, su front ya sabe **qué datos necesita**. Eso nos dice **qué APIs construir**.

Entidades = los sustantivos 📦

- `Libro`, `Autor`, ... ya las modelaron en la documentación del Bloque 1: "02 - Modelado de Dominio"
- Hoy **las verifican**: ¿siguen igual después de haber construido las pantallas?

Operaciones = los verbos ⚙️

- Qué hace cada pantalla con esos datos: `listar`, `ver`, `crear`, `editar`...
- Salen de **mirar su propio front**, nada nuevo, ya lo implementaron

¿Por qué diseñar antes de programar?
Porque arrancar a codear sin saber qué entidades, relaciones y endpoints necesitás es programar a ciegas.



OPERACIONES DE CADA PANTALLA

Recorran su app **pantalla por pantalla** y anoten qué necesitan hacer con los datos:

Pantalla	Recurso	Entidad
Catálogo	Listar libros	Libro
Catálogo	Buscar / filtrar	Libro
Detalle	Ver un libro	Libro
Alta	Crear un libro	Libro

- Cada fila es algo que **su front ya hace** (o maquetó) → nada que inventar
- La operación va en **acciones simples**: listar, ver, crear, editar, borrar, etc. (sin tecnicismos)
- La entidad **es la del modelo** que hicieron en la documentación del Bloque 1: "02 - Modelado de Dominio"



Taller: Modelo + Operaciones

Consigna, por grupo, para su TP:

1. **Verifiquen su modelo** (~5') en "**02 - Modelado de Dominio**": ¿están las entidades y atributos que terminaron usando en sus pantallas? Ajusten lo que falte.
2. **Listen las operaciones** (~25') en un nuevo documento "**05 - Diseño de API**": recorran la app pantalla por pantalla y anotá cada operación y sobre qué entidad.

Mecánica:

- 👥 En equipo, usan su propio repo y su front como referencia
- ⏱️ **Tiempo total**: ~35 min
- ✅ Objetivo: modelo verificado + tabla de operaciones con la pantalla/entidad principal
- 🎲 Presentar algunos ejemplos de cada equipo al resto de la clase

Actividades de la semana

Actividad Individual de la semana

Hasta el **12/07/2026**
(inclusive)
Única instancia

[AI-C13] Docker + encuesta

1. 🐳 ***Docker funcionando en librería:***
   - En el repositorio ds2026-actividades copiar el sitio de la librería de la Clase 11 en subdirectorio nuevo para esta clase: **ejercicios/c13-docker/**
   - *Instalar docker en la PC y crear el entorno básico backend (api + db) siguiendo los pasos indicados en la clase (tienen el branch C13 del repo de ejemplo como referencia). Commitear la configuración.*

2. 📋 ***Completar Encuesta de Cierre del Bloque 2:***
   *Ingresar y completar la **encuesta** de satisfacción del Bloque 2, es anónima y rápida, nos permite medir y mejorar el dictado de la materia.*

Comunicar la finalización en Discord en un mensaje dentro del canal **#actividades**, en el Hilo: "***[AI-C13] Docker + Encuesta Bloque 2***".

Actividad grupal

Hasta el **12/07/2026** (inclusive)

[AG-C13] Modelo de dato verificado + tabla de operaciones

Verificar el Modelo de datos y sumar la tabla de operaciones en el TP
(Completar y subir los elementos del taller iniciado en clase)

1. *Completar la verificación del modelo de dominio del Bloque 1: "**02 - Modelado de Dominio**" ¿están las entidades y atributos que terminaron usando en sus pantallas? Ajusten lo que falte.*
2. ***Completar y subir al repositorio la Listea de operaciones**, dentro de un nuevo documento en la carpeta "**Documentación**" > "**05 - Diseño de API**": recorran la app pantalla por pantalla y anoten cada operación y sobre qué entidad (como en la tabla de ejemplo).*
3. Comunicar la finalización en Discord en un mensaje único por equipo, dentro del canal **#actividades**, en el Hilo: "***[AG-C13] Modelo de dato verificado + tabla de operaciones***".



Antes de la próxima clase

⚠️ **Docker funcionando**

La práctica de C14 requiere tener el backend con docker.

Esta semana instalen Docker Desktop y completen la actividad individual.

🏖️ **Receso:** 20/07 al 01/08

La próxima clase (13/7) es la última clase antes